Doramagic v5

产品定义与技术规划全书

2026-03-15

汇总 ~15 轮三方研究，为开发做准备

Tang（产品决策）
Claude Opus（知识工程+架构）
Gemini（产品+市场+UX）
Codex（工程实现+算法）

不教用户做事，给他工具。
代码说事实，AI 说故事。
能力升级，本质不变。

第一部分：产品灵魂

1.1 一句话定义

Doramagic 是运行在 OpenClaw 上的哆啦A梦——用户说出模糊烦恼，Doramagic 从开源世界找最好作业，提取智慧，锻造开袋即食的 AI 道具。

1.2 产品设计之魂（不可修改）

不教用户做事，给他工具。

灵感来自哆啦A梦：哆啦A梦从不教大雄怎么做事，而是从口袋里掏出道具让大雄自己解决问题。Doramagic 是知识魔法（武装用户），而非方法论魔法（约束用户）。

1.3 三大架构原则

原则	含义	v5 验证
代码说事实，AI 说故事	确定性提取为骨架，LLM 仅做解读	Agent Harness 行业趋势完全吻合；Agentic 架构中 explora
能力升级，本质不变	每次新增能力都是叠加，不是替换	15 轮研究全部遵循：Stage 1.5 不替代 Stage 1，Health Che
不教用户做事，给他工具	输出 X 光片，不开处方	Health Check 不给总分；错误共识只标风险信号不判断对错

1.4 交付姿态（v4 确立）

Doramagic 永远交付完整的工作成果，scope 随覆盖度变化，但工作本身是完整的。明标边界，不自我矮化。

1.5 护城河

层级	护城河	竞品可复制性
短期	WHY + UNSAID 提取能力（行业空白）	中（好 prompt + 采样可逼近）
中期	跨项目知识关系引擎（公约数 + 打架 + 缝合）	低（需要积累数据和算法）
长期	处理不完整知识的能力 + 知识供应链溯源	极低（需要完整生态）

v5 新增：Agentic 提取让 Doramagic 成为 Cursor/Devin 的"上位知识供给者"——它们是建设者/修理工，Doramagic 是考古学家+哲学家。提取的灵魂可以喂给它们。

1.6 Agent Harness 验证（v5 新增）

2026 年 Agent Harness Engineering 行业趋势验证了 Doramagic 已有架构：

- "代码说事实，AI 说故事" = Harness Engineering 精髓
- Doramagic 在行业命名这个概念之前已实践
- 借鉴 4 个模式：Filesystem-as-Memory / Build for Deletion / Context Compaction Hierarchy / Entropy Management

1.7 gstack 灵魂提取启示（v5.1 新增）

对 Garry Tan（Y Combinator CEO）开源的 gstack 做灵魂提取后，得到三个工程纪律启示和两条"不该学"的边界：

学什么（工程纪律）

启示	说明	落地时机
SKILL.md 模板防腐系统	`.tmpl` 模板 + 代码元数据 → 自动生成 SKILL.md。结构性保证文档与代码不脱节。gstack 的工具名改了，	Phase 4
Tier 1 免费静态验证	解析 SKILL.md 中所有引用的命令/文件/Stage，验证它们是否真实存在。<5 秒，零成本，抓住 95% 低级错	Phase 4
Agent 友好的错误信息	错误信息写给 Agent 看，不是写给人看。每条错误告诉 Agent 该干什么。如：`File not found → R	随工具开发时采用

不学什么（产品边界）

不学	原因
gstack 的"有态度"面向用户	gstack 告诉用户"你应该怎么工作"。Doramagic 的灵魂是"不教用户做事，给他工具
gstack 的产品形态（Claude Code 插件集）	Doramagic 是独立的知识提取引擎，不是 Claude Code 的附属品。SKILL.md 是实

gstack 核心洞察的验证

gstack 证明了：AI Agent 的上限不取决于模型多强，而取决于你给它的认知框架多精确。 8 个 SKILL.md 文件 × 每个 200-500 行 > 任何 fine-tuning。

这与 Doramagic 的 Stage 架构高度一致——每个 Stage 本质上就是一种认知模式（Stage 1 = 广度扫描模式，Stage 1.5 = 假说验证模式，Stage 3.5 = 质量把关模式）。gstack 用"显式档位"（explicit gears）来表述这个理念，我们用"多阶段管线"表述，底层逻辑相同。

第二部分：用户视角

2.1 用户旅程一：发明工具（核心旅程，v4 确立）

Phase 1 需求挖掘 → Phase 2 发现作业 → Phase 3 提取灵魂 → Phase 4 锻造道具 → Phase 5 交付验证

这是 Doramagic 的核心价值链，所有其他旅程都是衍生。

2.2 用户旅程二：使用工具（核心旅程，v4 确立）

道具自解释、回炉改造、影子锻造、多道具协作

2.3 用户旅程三：领域探索（衍生能力，v5.1 降级）

v5.1 Tang 审阅修正：领域探索不是 Doramagic 的核心价值，是跨项目引擎能力的"无意中发明的应用场景"。不应与核心旅程并列，降级为衍生能力。不需要专门做三方用户场景研究。

阶段	用户行为	Doramagic 能力
领域真相	"告诉我 SEO 领域的真相"	领域共识图谱 → DOMAIN_TRUTH.md（铁律 + 争议 + 前沿）
项目体检	"给我的项目做体检"	知识健康检查 → HEALTH_CHECK.md（7 类信号定位）
项目对比	"这两个项目有什么区别"	跨项目智能 → SOUL_DIFF.md（共识 + 差异 + 路线分歧）
知识缝合	"把这几个项目的优点组合起来"	StitchCraft → STITCH_MAP（组合可能性地图 + 冲突标注）

2.4 用户角色矩阵与输出形态多样化（v5.1 更新）

v5.1 Tang 审阅修正：核心用户始终是开发者（抄作业的人）。其他角色是"无意中的扩展"。但终极思考是——Doramagic 的输出不必锁定为"只能是 skill"。跨项目引擎带来的知识服务（领域真相、体检报告、对比报告）对小白用户也有独立价值。输出形态多样化是能力的自然延伸，不是刻意设计新产品。

输出形态	消费者	优先级
CLAUDE.md / Skill	开发者（核心）	最高
热力图（知识差距可视化）	开发者 / 选型者	高（v5.2 新增）
DOMAIN_TRUTH.md	领域学习者 / 选型者	衍生
HEALTH_CHECK.md	项目作者 / 评估者	衍生
SOUL_DIFF.md	选型者 / 技术决策者	衍生
STITCH_MAP	高级开发者	衍生

角色	核心需求	最相关能力	优先级
开发者	抄作业、学架构、避坑	Soul Extractor（核心）	**核心**
项目作者	知识盲区检测 + 独创确认	Health Check	扩展
技术选型者	快速定位项目在领域中的位置	Health Check + 跨项目对比	扩展
创业者	寻找差异化点 + 壁垒评估	知识通胀 + 领域真相	扩展
投资人	套壳检测 + 创新度评估	跨项目对比	扩展
PM / CTO	技术资产盘点	企业级体检	远期

2.5 竞品定位（v5 新增）

竞品	做什么	Doramagic 的差异
Cursor / Copilot	建设者（帮你写代码）	Doramagic 提取的灵魂喂给它们
Devin / SWE-agent	修理工（帮你修 bug）	Doramagic 理解 WHY，它们理解 HOW
SonarQube	纠错别字（代码质量）	Doramagic 评估论文观点是否过时（知识质量）
Gartner / Tech Radar	分析师手工+年度/季度	Doramagic 自动从代码提取+持续更新
Google Scholar	论文引用（学术）	Doramagic 追踪逻辑实现（工程）

第三部分：核心机制

3.1 飞轮模型（v4，保持不变）

```
■■■■ → ■■ → ■■■■■ / ■■■■■■■■■ → ■ skill ■■ → ■■■■■■■■
```

3.2 第二飞轮：领域知识积累（v5 新增）

```
■■■■ → ■■■■■■■■ → ■■■■■■■ → ■■■■■■■■ → ■■■■
```

按领域深耕而非广撒网。第一个领域待定（需 2-3 个领域横向比较后决定）。

3.3 6 类知识对象（v4，保持不变）

Capability / Rationale / Constraint / Interface / Failure / Assembly Pattern

按知识类型组织提取，不是提取完再事后拆。

3.4 跨项目知识关系引擎（v5 新增）

3.4.1 核心能力

能力	描述	产出
项目指纹	机器可比较的项目知识表示	`project_fingerprint.json`
项目对比	两两或多项目知识差异分析	`SOUL_DIFF.md` / `CONSENSUS.md`
最大公约数	N 个同领域项目的共识知识提取	领域共识图谱
项目打架	原创 vs 套壳判断（fork 检测 + 知识层分析）	打架是公约数的前置步骤

3.4.2 三层知识表示 + 社区维度

v5.1 Tang 审阅修正：跨项目引擎要充分运用用户和社区信息。社区信息是 WHY 和 UNSAID 的富矿——代码说"怎么做的"，社区说"为什么这么做"和"踩了什么坑"。

层	内容	用途
Code Fingerprint	路径骨架 + 命令集 + 依赖集	代码层相似度（fork 检测）
Knowledge Atom	规范化的原子知识声明（subject/predicate/object/scope）	知识层匹配
Soul Graph	类型化节点 + 关系边	灵魂层对比
Community Signal	issue 热度 + PR 争议模式 + changelog 演化频率 + 社区情绪反馈	社区层对比（v5.1 新增）**

`project_fingerprint.json` 应包含社区维度：issue 活跃度、PR merge 速度、changelog 更新频率、社区情绪（正面/争议/冷清）。跨项目对比时，community_signals 应与代码/知识并列为第三维度。

3.4.3 匹配算法

三层匹配：lexical（MinHash）→ semantic（embedding ANN）→ structured（slot matching）

评分体系 4 维：Similarity / Lineage / Consensus / Uniqueness

3.4.4 假公约数防护

`support_independence` 是公约数系统的生死线。v5 扩展为三层防护：

层	机制	检测目标
项目级独立性	support_independence（现有）	多个项目 ≠ 多个独立来源
上游级独立性	provenance_independence（供应链 P1）	追溯到同一上游 = 回声，不是独立验证
认知级可靠性	consensus_risk（错误共识 MVP）	来源独立但都错了 = 最隐蔽的假公约数

3.5 领域共识图谱（v5 新增）

定位

图谱是飞轮的冷启动燃料，不是新产品。用户看到的不是图谱本身，而是图谱增强后的提取质量。

数据结构

- 基本单位：Atom Cluster（representative_atom + member_atoms + support_count + independence + scope_signature）
- 存储：SQLite + Parquet + JSONL
- 领域是视图，不是组织单位（知识原子池，按领域标签筛选）

注入点

Stage 0/1（最高杠杆，Brick 效应已验证 +20%）+ Stage 3.5（交叉验证）

图谱消费层

消费方式	输出
领域真相报告	DOMAIN_TRUTH.md（铁律 + 争议 + 前沿 + 暗雷）
知识健康检查	HEALTH_CHECK.md（7 类信号定位）
提取质量增强	DOMAIN_BRICKS.json（注入 Stage 0/1）
知识差距热力图	**GitHub vs 社区知识差距可视化（v5.2 新增）**

热力图验证结论（v5.2 新增）

三方调研（Gemini + Opus）验证了热力图在 Doramagic 中的定位和价值：

跨领域一致的知识差距：GitHub 代码知识 vs 社区实践知识存在稳定差距——WHY 维度差 ~6 分，UNSAID 维度差 ~5-7 分。这一差距跨领域一致，正是 Doramagic 提取的核心价值区间。

热力图本身就是产品输出：热力图不仅是内部优化工具，也是可以直接交付给用户的产品——让用户一眼看到"这个项目/领域的知识哪里密集、哪里空白"。

实现要点：

- 产品热力图（代码模块 x 知识类型）和实验热力图（项目 x 模型 x 版本）是两种不同数据，不要混淆
- 推荐 Treemap + 热力色做项目总览（面积=代码量，颜色=知识密度），Profile Heatmap 做单知识诊断
- 因果链不适合热力图，用 DAG/桑基图，点击热力图 drill down 到因果链
- 技术选型首选 ECharts（Treemap/Heatmap 原生支持），备选 D3.js

3.6 知识健康检查（v5 新增）

定位

领域图谱的镜子功能。GIS 叠加分析，不是医疗体检/财务审计/代码审查。

7 类信号

信号	确定性	处理方	描述
ALIGNED	高	代码	项目知识与领域共识一致
STALE	高	代码	使用了已废弃/过时知识
MISSING	中	代码+AI	领域共识中存在但项目缺失
ORIGINAL	中	代码+AI	项目独有，图谱中未出现
DIVERGENT	低	AI	项目与领域存在路线分歧
DRIFTED	中	代码+AI	共识已演化但项目未跟进
CONTESTED	低	AI	领域内尚无共识，多路线并存

工程双路分离

- Route A（确定性）：ALIGNED / STALE / MISSING / ORIGINAL — 靠匹配和阈值
- Route B（解释性）：DIVERGENT / DRIFTED / CONTESTED — 需要更多上下文

匹配算法

```
MatchScore = 0.30 × subject + 0.25 × predicate + 0.15 × object + 0.20 × scope + 0.10 × normative_force
```

四段阈值：EXACT_ALIGNED ⩾ 0.92 / SEMANTIC ⩾ 0.80 / PARTIAL ⩾ 0.62 / NO_MATCH < 0.62

MISSING 双阈值

coverage ⩾ 0.60 + independence ⩾ 0.55 + 独立支持 ⩾ 3

ORIGINAL 二次检验

更大候选窗口重新检索，确认仍无匹配才标 ORIGINAL。防"表述不同"假阳性。

6 个可观测维度（不定义"健康"，不给总分）

共识覆盖率 / 时效性 / 独创密度 / 路线位置 / 缺失区域 / 参照系强度

情感设计

禁止"错误"/"漏掉"/"落后"。推荐"观测到差异"/"非主流路径"/"新兴趋势"。

STALE 独立运行

STALE 不依赖图谱，只依赖 `deprecation_events.jsonl`。是体检的最小可用子集，最早可上线。

3.7 StitchCraft：知识缝合（v5 新增）

定位

缝合 ≠ 自动组装。输出是"组合可能性地图"（STITCH_MAP），不是"缝好的 skill"。Doramagic 是制版师（给裁剪图和面料清单），不是裁缝。

6 类冲突

semantic / scope / architecture / dependency / operational / license

冲突处理原则

事实冲突确定性解决 / scope 冲突标注 scope / 路线冲突展示帕累托前沿

输出

STITCH_REPORT.md + STITCHED_KNOWLEDGE.md + ASSEMBLY_PLAN.md + LICENSE_NOTICE.md

Stitch Validation Gate

6 项检查：Consistency / Dependency closure / Assumption closure / License closure / Traceability / Actionability

3.8 知识供应链 + 通胀检测（v5 新增）

三种时间衰变模式

模式	定义	检测方式
STALE	知识曾对，现被官方废弃	废弃事件数据库
DRIFTED	知识重心在领域演化中偏移	同一主题权重/排序变化
INFLATED	知识仍对，但不再稀缺	跨项目渗透率递增

通胀度量

```
InflationIndex = penetration_rate(now) / penetration_rate(first_seen)
```

四阶段：Frontier(<10%) → Diffusion(10-50%) → Consensus(50-90%) → Infrastructure(>90%)

二阶知识通胀（关键洞察）

一阶通胀但深度理解仍稀缺。例：

- 一阶："FAQPage 受限" → 100% 通胀
- 二阶："FAQPage 对 AI 仍有语义价值" → ~50%
- 三阶："三档分级处理策略" → <25%

normative_force 作为零成本扭曲预测器

描述性知识传播零扭曲，规范性知识传播扭曲大。

6 种传播变形模式

范围扩大化 / 去上下文化 / 时间冻结 / 对立面合并 / 权威借用 / 零扭曲传播（fork）

5 种上游源头分类（确定性递减）

平台官方公告 > 学术/行业实证 > 行业分析机构 > 社区经验沉淀 > 个人实践推断

NEVER 约束

永远不做"硬判定唯一源头"。开源知识传播天然隐式。

3.9 错误共识检测（v5 新增）

定位

假公约数的终极形态——来源独立、上游不同，但结论一样且都错了。Tier-0 暗雷（系统级感染）。

5 种子类型

共同认知偏差 / 过时共识 / 数据源错误 / 正确数据+错误解读 / 幸存者偏差

7 个风险信号

#	信号	精度估算
S1	Evidence-Assertion Gap（强断言弱证据）	~0.5-0.6
S2	Temporal Freeze（时间冻结）	~0.6-0.7
S3	Indirect Evidence Only（无直接代码证据）	~0.5
S4	Interpretation Escalation（相关性→因果性跳跃）	~0.6
S5	Contradicts Official（与官方矛盾）	~0.7-0.8
S6	Single-Study Dependency（单一研究依赖）	~0.6-0.7
S7	Implementation Gap（声称功能未实现）	~0.6

组合规则

1 信号 = CONTEXT / 2 信号 = SIGNAL / 3+ 信号 = HIGH-SIGNAL

术语约束

永远不说"错误共识"/"这是错的"。用"高风险共识"/"认知脆弱性"。

风险率估算

~4-6% 的 ALIGNED 知识原子

3.10 记忆系统（v4，保持不变）

v1 最简方案：文件即存储，LLM 即编译器。拒绝过度工程化。

3.11 多项目提取→组装管线（v5.2 新增）

定位

单项目管线回答："这个项目的灵魂是什么？"

多项目管线回答："用户想做 X，应该站在哪些前人的肩膀上？"

多项目管线调用单项目管线，不替代它。这是 Doramagic 从"提取引擎"升级为"端到端道具锻造"的关键一步。

与单项目管线的关系

```
■■■■■■■V5 ■■■■
Stage 0 → 1 → [1.5] → 2 → 3 → 3.5 → 4 → 5 → CLAUDE.md

■■■■■■■v5.2 ■■■■
Phase A → B → C■■■■■■■■■×N■ + D → E → F → G → H → SKILL.md
↑
■■■■■■■■■■■
```

完整管线（Phase A-H）

```
Phase A: ■■■■
↓
Phase B: ■■■■■■■■+■■+■■■
↓
Phase C: ■■■■■■■■N ■■■■■■■■■■■■
↓
Phase D: ■■■■■■■■ClawHub/■■/■■■
↓
Phase E: ■■■■■■■■■+■■+■■■■■■
↓
Phase F: Skill ■■■■■■■■+■■■■■+■■■■■+■■■■■■■■
↓
Phase G: ■■■■■■Validation Gate■
↓
Phase H: ■■■SKILL.md + ■■■
```

Phase A：需求理解

输入：用户的自然语言描述（如"我想做一个记录食物卡路里的 skill"）

处理：

- 提取关键词和意图（LLM 理解）
- 分类需求类型：功能需求 / 场景约束 / 质量期望
- 生成搜索计划：要搜哪些方向？优先级？

输出：`need_profile.json`（含 raw_input / keywords / intent / search_directions / constraints）

执行者：LLM（OpenClaw agent 或主对话），< 1 分钟

开发注意事项：

- `need_profile.json` 的 constraints 必须包含 `"OpenClaw ■■"` 约束，驱动后续 Phase F 的平台适配
- search_directions 的 priority 字段驱动 Phase B 的搜索顺序，不要遗漏 "社区已有 skill" 方向

Phase B：作业发现

输入：`need_profile.json`

处理：按 search_directions 并行搜索（GitHub API + ClawHub + 社区资源）→ 粗筛（stars/更新/README/暗雷黑名单）→ 精筛（相关度/质量信号/知识互补性）→ 推荐 Top 3-5

输出：`discovery_result.json`（含 candidates[] + search_coverage）

执行者：搜索工具 + LLM 判断，2-5 分钟

开发注意事项：

- 粗筛阈值（stars > 10 / 6 个月内更新）是初始值，需根据领域调整（小众领域可能需要降低）
- 知识互补性评估是精筛的关键——已选项目覆盖了什么，还缺什么
- 发现结果要区分 `github_repo`（走 Phase C 完整提取）和 `community_skill`（走 Phase D 轻量提取）

Phase C：并行灵魂提取

输入：`discovery_result.json` 中的 GitHub 项目列表

处理：对每个 GitHub 项目走单项目管线（Stage 0→1→[1.5]→2→3→3.5→[4]→5）

关键约束：

- 必须走 Stage 0（确定性提取 repo_facts.json）
- 必须走 Stage 3.5（证据绑定，claim 必须有 file:line）
- Stage 1.5（Agentic Loop）视项目规模决定是否启用

- N 路并行，每路独立子代理

输出：每个项目一个标准提取包（repo_facts.json / soul_discovery.md / concept_cards / workflow_cards / decision_rule_cards / validation_report.json / expert_narrative.md / CLAUDE.md）

执行者：N 个并行子代理（Sonnet/Opus），每个项目 5-15 分钟

开发注意事项：

- 并行子代理之间完全隔离，不共享中间状态
- 提取包的目录结构必须统一，Phase E 依赖此结构做批量读取

Phase D：社区知识采集

输入：`discovery_result.json` 中的社区 skill / 教程 / 用例

处理：对每个社区资源执行轻量提取（不走完整 Stage 0-5）：读取 SKILL.md / 教程内容 → 提取核心功能、触发条件、工具调用、数据格式、存储路径 → 转化为结构化知识卡片

与 Phase C 的区别：Phase C 提取"代码项目的灵魂"（深度，Stage 0-5），Phase D 提取"已有 skill 的经验"（宽度，轻量读取）

输出：`community_knowledge.json`（含 skills[] / tutorials[] / use_cases[]）

执行者：1 个子代理（Sonnet），2-5 分钟

开发注意事项：

- 社区 skill 的 SKILL.md 格式可能不规范，需要容错解析
- 重点提取可复用知识：存储路径标准、提醒时间、日志格式、工具调用模式

Phase E：知识综合

输入：Phase C 的 N 个提取包 + Phase D 的社区知识

处理：公约数提取 → 打架分析 → 独创标注 → 与用户需求匹配

输出：`synthesis_report.md`（共识知识列表 + 冲突列表 + 独创知识列表 + 知识选择建议）

执行者：Opus 子代理（需要深度理解和权衡），3-5 分钟

开发注意事项：

- 综合报告的每条知识必须标注来源项目和证据强度
- 冲突分类复用 StitchCraft 的 6 类冲突体系（semantic / scope / architecture / dependency / operational / license）
- 知识选择建议要说明"为什么纳入"和"为什么排除"，不能只列结果

Phase F：Skill 组装 + 平台适配（v5.2 含 OpenClaw 适配检查）

输入：`synthesis_report.md` + `need_profile.json`

处理：

- 知识选择：从综合报告中选择纳入 skill 的知识
- 冲突解决：事实冲突选证据更强的；路线冲突展示选项或选与需求更匹配的
- 格式编译：编译为 OpenClaw SKILL.md 格式（frontmatter / 工作流 / 数据存储 / AI prompt 契约 / 人格配置 / cron 提醒）
- 平台适配检查（v5.2 新增，详见 4.11）

输出：`SKILL.md`（可直接部署到 OpenClaw 的 skill 文件）

执行者：LLM + 确定性模板，3-5 分钟

开发注意事项：

- 平台适配检查必须在输出 SKILL.md 之前执行，不能事后补救
- cron 不在 SKILL.md frontmatter 中配置，需要通过平台其他机制设置（见 4.11）

Phase G：质量门控

输入：`SKILL.md` + `synthesis_report.md`

检查项（从 StitchCraft Validation Gate 扩展）：

#	检查项	说明	阻断？
1	**Consistency**	skill 内部知识是否一致（无自相矛盾）	是
2	**Completeness**	用户需求中的每个功能是否都被覆盖	是
3	**Traceability**	每条知识是否可追溯到来源项目	是
4	**Platform Fit**	是否符合 OpenClaw SKILL.md 格式规范	是
5	**Conflict Resolution**	所有冲突是否已解决（无未处理冲突）	是
6	**License**	所有来源的许可证是否兼容	警告
7	**Dark Trap Scan**	是否引入了已知暗雷（Top 10 暗雷扫描）	警告

输出：`validation_report.json`（PASS / REVISE / BLOCKED）

执行者：确定性检查 + LLM 辅助，1-2 分钟

开发注意事项：

- Platform Fit 检查包含 4.11 中定义的所有平台适配规则
- REVISE 时自动反馈给 Phase F 重新组装，最多 2 轮

Phase H：交付

输入：通过门控的 `SKILL.md`

处理：输出 SKILL.md + 使用说明 + 知识溯源摘要 + 已知限制

输出：交付包（SKILL.md / README.md / PROVENANCE.md / LIMITATIONS.md）

开发注意事项：

- PROVENANCE.md 必须列出每个来源项目的 URL 和许可证
- LIMITATIONS.md 从 Phase G 的 Dark Trap Scan 结果中提取

第四部分：技术架构

4.1 架构总览

两层分离（v4，保持不变）

提取核心（平台无关）+ 平台适配层（可替换）

完整管线（v5 更新）

```
Stage 0: ■■■■■■■prepare-repo.sh + repomix + repo_facts.json + community_signals■
↓
Stage 1: ■■■■■■■■■■ Q1-Q7 + v5 ■■ Q8 ■■■■■■
↓
[NEW] Stage 1.5: Agent Exploration Loop■■■■■■■■■■■
↓
Stage 2: ■■■■■■concept_cards + workflow_cards■
Stage 3: ■■■■■■decision_rule_cards■
↓
Stage 3.5: ■■■■■■■fact-checking + v5 ■■ claim traceability■
↓
[■■] Stage 3.7: ■■■■■■■■■■■■■■■■■■■■
↓
Stage 4: ■■■■■■■■■■■
↓
Stage M: ■■■■■
Stage C: ■■■■■
Stage F: ■■■■■■CLAUDE.md■
```

4.2 Stage 0：确定性提取

保持不变：prepare-repo.sh（四层漏斗采样）+ repomix + extract_repo_facts.py → repo_facts.json

v5 扩展：

- `repo_facts.json` 三重角色：初始知识 + 假说种子 + fact-checking 基线
- 预留 `upstream_references[]` 字段（供应链 P0）

4.3 Stage 1：广度扫描

v5 扩展：新增 Q8 —— "列出 3-5 个你最想深入了解的问题/假说"

输出：`hypothesis_list.json`（假说列表，驱动 Stage 1.5）

4.4 Stage 1.5：Agent Exploration Loop（v5 核心新增）

架构

混合架构（Stage 0 广度 + Agent 深度）。不替代任何现有 Stage。

核心原理

Agentic 提取解决的是"输入信息不足"，不是"处理深度不足"。Session 24-25 数据：Stage 0 改进 +0.7，Prompt 改进 +0.3。

A/B 验证结论（v5.2 新增）

Agentic Stage 1.5 已完成 A/B 对照验证，两组实验揭示了广度与深度的互补关系：

维度	A 组（单遍扫描）	B 组（假说驱动）
知识条数	46 条	20 条
证据精度	文件名级	file:line 级
覆盖特征	广但浅	窄但极深

核心结论：

- A 和 B 是互补关系，不是替代关系——验证了 Stage 1（广度扫描）+ Stage 1.5（假说深挖）的双阶段设计正确性
- Agentic 的真正价值不在于"发现更多"，而在于：挖得更准、能推翻错误直觉、能精确到行号
- 假说质量是 Stage 1.5 的真正瓶颈——低质量假说浪费 Agent 算力，高质量假说才能触发深层发现

开发注意事项：

- Stage 1 的 Q8 假说生成质量直接决定 Stage 1.5 的 ROI，需要投入精力优化假说生成 prompt
- A 组结果可作为 Stage 1.5 的 baseline——如果 Agent Loop 的产出不比 A 组多出有意义的深度知识，说明假说质量不够
- 考虑在 Stage 1.5 中增加假说质量自评环节：Agent 每轮开始前评估剩余假说的预期信息增益

5 个工具

工具	用途
`read_artifact`	读取 Stage 0/C 的结构化产物
`list_tree`	列目录树（带过滤）
`search_repo`	ripgrep 式搜索
`read_file`	读取文件片段
`append_finding`	结构化记录发现

5 个中间文件（Filesystem-as-Memory）

文件	作用
`hypotheses.jsonl`	Stage 1 产出的待验证假说
`exploration_log.jsonl`	每次工具调用的事实记录
`claim_ledger.jsonl`	已确认/已反驳/待定的知识声明
`evidence_index.json`	证据→文件→行号→假说的索引
`context_digest.md`	压缩后的当前理解（给下一轮 fresh context）

交互模式

短轮次交互：每轮 fresh context + 读文件记忆。不靠长对话记忆，靠文件记忆。

三层 Budget

类型	机制
硬 budget	工具调用次数上限
软 budget	token 总量上限
信息增益	连续 N 轮无新发现则停止

弱模型降级

MiniMax 等弱模型：增强版 Stage 0（Haiku 辅助采样），跳过 Agent Loop。注入错误知识比不注入更危险。

降级路径

Agent Loop 失败 → 回退到纯 Stage 1 输出，不比现在差。

收益预测

- 第 1-3 轮：60% 价值（核心假说验证）
- 第 4-7 轮：30%（边缘假说追踪）
- 第 8+ 轮：10%（收益递减）
- 默认：小项目 5 轮，大项目 10 轮

成本

~2x 增幅。python-dotenv $0.06→$0.11，wger $0.17→$0.31。

Build-for-Deletion

假说 prompt / 工具调用上限 / exploration_log 强制记录 / INFERENCE 标记 — 都应可插拔，模型进化后可移除。

Agent 友好错误信息（v5.1 新增，源自 gstack 启示）

Stage 1.5 的 5 个工具返回错误时，必须告诉 Agent 下一步该干什么：

错误类型	不要这样	应该这样
文件不存在	`FileNotFoundError: /src/foo.py`	`File not found at /src/foo.py. Run list_tree("src/") to see a
搜索无结果	`No matches found`	`No matches for "auth". Try broader terms like "login" or "s
读取超长	`Content too large`	`File has 5000 lines. Use read_file with line range, e.g., rea

4.5 Stage 3.5：验证硬阻断（v5 增强）

现有：structured_feedback.json + retry loop + fact-checking

v5 新增：

- `check_claims_have_evidence()`：Stage 1.5 的 claim 必须引用 exploration_log entry
- 无证据的 claim 标记为 `INFERENCE`，不参与 Stage 4 叙事的"设计哲学"部分

4.6 Stage 3.7：知识健康检查注入（v5 新增，可选）

触发条件：检测到匹配领域图谱 + 图谱达到 Beta 规模（8+ 项目）

产出：`health_check.json` + `HEALTH_CHECK.md`

4.7 知识供应链基础设施（v5 新增）

数据结构

结构	用途
`SourceRef`	上游来源元数据（URL/publisher/published_at/confidence）

结构	用途
`AtomOccurrence`	知识在项目中的出现位置
`ProvenanceEdge`	引用关系（from→to + confidence + mode）
`PropagationEdge`	项目间传播关系
`ProvenanceCluster`	同源聚类

三层推断置信度

层级	精度	召回	用途
explicit_confirmed	0.90-0.98	0.15-0.35	进入事实层
inferred_high_precision	0.60-0.80	0.30-0.50	进入分析层，需标记
inferred_speculative	0.35-0.60	0.50-0.70	仅候选，不进报告

independence 升级

```
{
"support_independence": 0.85,
"provenance_independence": 0.30,
"upstream_authority": "official_announcement",
"interpretation_variance": 0.15,
"normative_force_adjustment": 0.92
}
```

4.8 废弃事件数据库（v5 新增）

`deprecation_events.jsonl`：独立的 append-only 知识库

三级来源：official（最高优先级）> graph observations > community consensus

可在无领域图谱时独立运行 STALE 检测。

4.9 子代理架构（v4+v5）

模型路由表 + 模型能力抽象层（不绑定品牌）

v5 补充：

- Opus 子代理：架构决策、产品分析、代码审查、复杂 debug
- Sonnet 子代理：代码搜索、文件探索、批量编辑、简单代码生成
- Haiku 子代理：文件查找、状态检查、简单数据提取

4.10 对外接口

API 是一等公民，MCP 是可选适配层。

4.11 OpenClaw 平台适配规则（v5.2 新增）

背景

SKILL.md 格式验证发现 OpenClaw 平台与 Claude Code 存在多处差异。Phase F（Skill 组装）和 Phase G（质量门控）必须内置平台适配检查，否则生成的 SKILL.md 无法在 OpenClaw 上运行。

已知适配规则

项目	Claude Code	OpenClaw	处理方式
allowed-tools 名称	Bash / Read / Write	exec / read / write	Phase F 编译时自动映射
metadata.openclaw 字段	自由格式	仅支持 always / emoji / homepage / os / primaryEnv / requires / install	Phase F 编译时校验，移除不支持字段
cron 配置	可在 frontmatter 声明	不在 SKILL.md frontmatter 中配置，通过平台设置	Phase F 编译时从 frontmatter 移除 cron，输出到

适配检查清单（集成到 Phase G 的 Platform Fit 检查项）

- 工具名映射：扫描 SKILL.md 中所有 `allowed-tools`，验证是否使用 OpenClaw 命名
- metadata 字段校验：扫描 `metadata.openclaw` 下所有字段，标记不在白名单中的字段
- cron 分离：确认 frontmatter 中无 cron 配置
- 路径规范：确认存储路径使用 `~/clawd/` 前缀（OpenClaw 标准）
- 工具引用一致性：确认 SKILL.md 正文中引用的工具名与 frontmatter 声明一致

实现要点

- 适配规则应抽象为配置文件（`platform_rules.json`），不硬编码在 Phase F 逻辑中
- 随 OpenClaw 平台更新，只需更新配置文件，不改代码
- Tier 1 静态验证（skill_check.py）应包含平台适配检查作为子集

第五部分：质量保证体系

5.1 四层门控（v4，保持不变）

需求完整性 / 源泉充分性 / 提取质量 / 道具可用性

5.1.1 SKILL.md 模板防腐系统（v5.1 新增，源自 gstack 启示）

问题：SKILL.md 手工维护，与代码脱节。工具名改了但 SKILL.md 没更新 → Agent 调用不存在的工具。

方案：

```
SKILL.md.tmpl■■■■■■■ + ■■■■■
↓
skill_gen.py■■■■■■■■■■■■Stage ■■■■■■■■■
↓
SKILL.md■■■■■■■■■■ Git■
```

- 模板包含工作流、提示、示例（需要人类判断的部分）
- `{{TOOL_REFERENCE}}` / `{{STAGE_LIST}}` / `{{OUTPUT_FORMAT}}` 等占位符由代码元数据填充
- CI 可验证新鲜度：`skill_gen.py --dry-run` + `git diff --exit-code`

落地时机：Phase 1 开工时引入。

5.1.2 Tier 1 免费静态验证（v5.1 新增，源自 gstack 启示）

问题：当前只有 Benchmark（贵、慢），没有"免费、秒级"的低级错误检查。

方案：`skill_check.py`，<5 秒，零成本：

- 解析 SKILL.md 中所有引用的文件路径 → 验证文件存在
- 解析所有引用的 Stage 名称 → 验证 Stage 定义存在
- 解析所有引用的工具名称 → 验证工具在注册表中
- 解析所有引用的输出格式 → 验证 JSON Schema 存在
- OpenClaw 平台适配检查（v5.2 新增）→ 验证 allowed-tools 命名 + metadata 字段 + cron 分离

三层测试体系（整合 gstack 启示）：

层	成本	速度	做什么
Tier 1 静态验证	免费	<5s	SKILL.md 引用验证 + 格式检查 + 平台适配检查
Tier 2 Benchmark	~$0.5-2	~10min	A/B 对照 + WHY Rubric 评分
Tier 3 LLM-as-judge	~$0.15	~30s	Sonnet 评分（清晰度/完整性/可操作性）

原则：95% 的问题免费发现，LLM 只用于需要判断力的场景。

落地时机：Phase 1 开工时引入。

5.2 幻觉三层防护（v4+v5 增强）

层	v4	v5 新增
第一性原理约束	repo_facts.json whitelist	exploration_log 证据绑定
社区证据交叉验证	community_signals.md	provenance clustering 同源检测

层	v4	v5 新增
项目规模调节	大/小项目策略	弱模型降级（跳过 Agent Loop）

5.3 知识质量多维评估（v5 新增）

维度	机制
正确性	fact-checking gate + deprecation_events
独立性	support_independence + provenance_independence
时效性	STALE / DRIFTED / INFLATED 三种衰变检测
可靠性	consensus_risk（错误共识风险信号）
独创性	ORIGINAL 信号 + likely_inflated 校正
溯源性	upstream_references + evidence_level

5.4 暗雷体系（v4+v5）

Top 10 暗雷：LLM 过度推理 > 隐性规模假设 > 架构考古 > 开源包闭源魂 > Hidden Context > 维护者独白 > Winner's History > Exception Bias > 简历驱动开发 > 幽灵约束

v5 新增：错误共识 = Tier-0 暗雷（系统级感染，高于所有单项目暗雷）

3 个管线防护：WHY 可恢复性判断 / 社区规则适用域约束 / Stage 3.5 暗雷审查

暗雷叠加效应：多暗雷同时命中，风险指数增长。

第六部分：开发路线与实施计划

6.1 架构依赖链（按此顺序构建，v5.2 更新）

```
1. project_fingerprint.json schema■■■■
2. compare_projects.py■■■■■■■■
3. Stage 1.5 Agentic Loop■■■■■■■■■
4. ■■■■ Alpha■SEO/GEO■7 ■■■
5. ■■■■■■ V0-V1
6. ■■■ P1■■■■■■ + provenance clustering■
7. StitchCraft V1
8. ■■■■ MVP
9. ■■■■■ Phase A-H ■■■■v5.2 ■■■
```

v5.2 注：多项目管线（Phase A-H）是上述 1-8 的"组装层"。Phase C 调用单项目管线（含 Stage 1.5），Phase E 调用跨项目基础设施（compare_projects.py），Phase F 调用 StitchCraft 的冲突解决。因此多项目管线排在最后，但其中各 Phase 的开发可以随上游组件就绪而逐步推进。

6.2 分阶段实施计划

Phase 0：基础设施（P0，零成本预备）

任务	工时	依赖	来源
atom schema 预留 `upstream_references[]`	0.5 天	无	供应链研究
independence 加入 normative_force 调节因子	0.5 天	无	供应链研究
SKILL.md 模板系统搭建（.tmpl + gen.py）	0.5 天	无	gstack 启示
skill_check.py 静态验证脚本（含 OpenClaw 适配检查）	0.5 天	无	gstack 启示 + OpenClaw 适配
`platform_rules.json` OpenClaw 适配规则配置	0.5 天	无	v5.2 OpenClaw 适配

Phase 1：Agentic 提取 MVP

任务	工时	依赖
Stage 1 新增 Q8 假说生成	0.5 天	无
Stage 1.5 Agent Loop（脚本 + 5 工具 + 5 中间文件）	3-4 天	Q8
Stage 3.5 claim traceability 增强	0.5 天	Stage 1.5
A/B 验证（3 项目）	1-2 天	上述全部
合计	**5-7 天**	

Phase 2：跨项目基础设施

任务	工时	依赖
project_fingerprint.json schema 定稿	1-2 天	无
compare_projects.py（三层匹配）	2-3 天	fingerprint
4 个 SEO 项目 gold set 验证	1 天	compare
合计	**4-6 天**	

Phase 3：领域图谱 Alpha

任务	工时	依赖
用 4 个 SEO 项目构建图谱原型（SQLite）	1-2 天	Phase 2
策展+提取 3-4 个补充 SEO 项目	1 周	原型
DOMAIN_BRICKS.json + DOMAIN_TRUTH.md 编译	1-2 天	Alpha 图谱
A/B 验证（有/无图谱注入）	1 天	编译

Phase 4：知识健康检查 V0-V1

任务	工时	依赖
V0：STALE 检测器 + deprecation_events.jsonl	1 天	无（可提前做）
scope ontology 固定	1 天	Phase 2 经验
V1-a：ALIGNED/MISSING/ORIGINAL 检测	2 天	领域图谱 Alpha
V1-b：health_check.json + HEALTH_CHECK.md 渲染	1 天	V1-a
V1-c：gold set 验证	1 天	V1-b
合计	**~6 天**	

Phase 5：供应链 P1

任务	工时	依赖
显式来源提取器	1-2 天	Phase 0
provenance clustering + adjusted_independence()	1-2 天	来源提取
ORIGINAL likely_inflated 弱校正	1 天	Health Check V1
合计	**3-5 天**	

Phase 6：StitchCraft V1

任务	工时	依赖
stitch_profile.json	1 天	Phase 2
conflict_graph.json + 冲突检测	1-2 天	stitch_profile
stitch_compiler.py + Validation Gate	1-2 天	conflict_graph
验证	1 天	compiler
合计	**4-6 天**	

Phase 7：错误共识 MVP

任务	工时	依赖
4 个核心风险信号检测	1-2 天	供应链 P1
consensus_risk 标注集成	0.5 天	信号检测
合计	**1-2 天**	

Phase 8：多项目管线自动化（v5.2 新增）

任务	工时	依赖	说明
Phase A 需求理解模块	1 天	无	`need_profile.json` 生成，可提前开发
Phase B 作业发现模块	2-3 天	Phase A	GitHub API + ClawHub 搜索 + 好作业评分

任务	工时	依赖	说明
Phase C 并行提取编排	1-2 天	Phase 1	调用单项目管线 x N，并行子代理管理
Phase D 社区知识采集	1-2 天	无	轻量提取 + `community_knowledge.json` 生成
Phase E 知识综合	2-3 天	Phase 2 + Phase 6	调用 compare_projects.py + StitchCraft 冲突体
Phase F Skill 组装 + 平台适配	2-3 天	Phase E + 4.11	SKILL.md 编译 + OpenClaw 适配检查
Phase G 质量门控	1-2 天	Phase F	7 项检查 + 自动 REVISE 循环
Phase H 交付包生成	0.5 天	Phase G	SKILL.md + README + PROVENANCE + LIMITATI
端到端集成测试	2-3 天	上述全部	用 Sim2（卡路里 v2）场景跑通全管线
合计	**13-19 天**		

开发注意事项：

- Phase A/D 无上游依赖，可与 Phase 1-7 并行开发
- Phase C 依赖 Phase 1（Agentic 提取 MVP），是关键路径
- Phase E 依赖 Phase 2（跨项目基础设施）和 Phase 6（StitchCraft），是最晚可开始的 Phase
- 端到端集成测试使用 Sim2（卡路里 v2）场景，因为该场景已严格按管线执行并通过 7/7 质量门控

总工时估算（v5.2 更新）

Phase	工时	可并行
P0 基础设施	1.5 天	—
P1 Agentic 提取	5-7 天	与 P2 部分并行
P2 跨项目基础设施	4-6 天	与 P1 部分并行
P3 领域图谱	~2 周	P2 完成后
P4 健康检查	~6 天	P3 完成后
P5 供应链	3-5 天	与 P4 并行
P6 StitchCraft	4-6 天	P2 完成后
P7 错误共识	1-2 天	P5 完成后
P8 多项目管线	**13-19 天**	**Phase A/D 可提前；Phase E 最晚**

第七部分：实验成果索引

7.1 已完成实验

实验	结论
exp01 v0.3 python-dotenv	灵魂提取有效（42%→96%）
exp02 v0.5 MiniMax	全 4 Stage 通过，跨模型可行
exp05 v0.8 superpowers	叙事范式转变验证
exp06 v0.9 benchmark	三路 A/B/C 对照（26/28/26）
exp07 MiniMax 用户测试	幻觉根因分析，证据绑定必要性
exp08 P0 patch + wger	traceability 100%，跨领域适应性
exp-seo-skills 4 项目	跨项目智能发现，4 份 CLAUDE.md
Session 24-25 WHY 实验	投资优先级排序，Agentic #1
Session 27 KC A/B	Knowledge Compiler 降级为工程任务
Session 27 UNSAID exp	核心壁垒确认，覆盖率与熟悉度反相关
Agentic Stage 1.5 A/B（v5.2）	A 组 46 条广浅 vs B 组 20 条窄深，互补关系验证，假说质量是瓶颈

7.2 模拟验证（v5.2 新增）

三轮模拟验证了多项目管线的设计正确性和通用性：

模拟	场景	结论
Sim1（卡路里 v1）	首次尝试"做一个记录卡路里的 skill"	暴露 5 个结构性缺口（需求理解缺失 / 搜索无系统 / 提取不并
Sim2（卡路里 v2）	严格按 Phase A-H 管线执行同一场景	7/7 质量门控全部通过。验证管线结构完整，每个 Phase 的输
Sim3（机票 skill）	跨领域验证——用管线做"机票搜索比价" skill	管线通用性确认。Phase A-H 的流程不需要领域特定修改

核心结论：管线是通用的，知识是领域特定的。Phase A-H 的流程架构不随领域变化，变化的是 Phase B 的搜索方向、Phase C 的提取项目、Phase D 的社区资源。

开发注意事项：

- Sim1 暴露的 5 个缺口已全部被 Phase A-H 覆盖，但实际开发中可能暴露新的缺口
- Sim2 的通过记录可作为端到端集成测试的 golden case
- Sim3 验证了管线不需要"领域模板"机制——通用管线 + 领域特定的搜索/提取即可

7.3 待做实验

实验	依赖	优先级
exp09 端到端验证	Phase 1-2	最高
Agentic A/B（3 项目，工程实现后）	Phase 1	高
领域图谱注入 A/B	Phase 3	高
多领域横向比较（2-3 新领域）	Phase 3	中
UNSAID Exp-02/03	Phase 1	中
多项目管线端到端（真实场景）	Phase 8	高（v5.2 新增）

第八部分：三方研究索引

#	课题	日期	综合报告位置
1	部分覆盖	2026-03-14	docs/research/20260314_doramagic_partial_c
2	好作业评判	2026-03-15	docs/research/20260315_good_homework_se
3	暗雷体系	2026-03-15	docs/research/20260315_misleading_good_ho
4	跨项目智能	2026-03-15	research/cross-project-intelligence/synthesis-re
5	StitchCraft	2026-03-15	research/cross-project-intelligence/stitchcraft-
6	领域共识图谱	2026-03-15	research/cross-project-intelligence/domain-m
7	知识健康检查	2026-03-15	research/cross-project-intelligence/health-che
8	知识供应链+通胀	2026-03-15	research/cross-project-intelligence/supply-cha
9	错误共识检测	2026-03-15	research/cross-project-intelligence/false-cons
10	Agentic 提取	2026-03-15	research/agentic-extraction/synthesis.md
11	Agent Harness	2026-03-15	Memory: agent-harness-analysis.md
12	热力图调研	2026-03-13	Memory: heatmap-research.md
13	WHY 提取实验	2026-03-13	Memory: why-extraction-conclusions.md
14	UNSAID 实验	2026-03-14	Session 27 chat log
15	KC A/B 实验	2026-03-14	Session 27 chat log
16	gstack 灵魂提取	2026-03-16	Session 30 chat log（garrytan/gstack）
17	多项目管线设计	2026-03-17	docs/multi-project-pipeline.md
18	Agentic Stage 1.5 A/B 验证	2026-03-17	v5.2 Section 4.4
19	OpenClaw 平台适配	2026-03-17	v5.2 Section 4.11
20	三轮模拟验证（Sim1/2/3）	2026-03-17	v5.2 Section 7.2

附录

A. 硬性约束清单

- 不教用户做事：所有输出是工具/信息，不是建议/指令
- 代码说事实：确定性提取为骨架，LLM 仅做解读
- 能力升级本质不变：新能力叠加，不替代
- 吃透 OpenClaw 规则：Doramagic 适应平台，不是反过来
- 偏向漏报不偏向误报：所有不确定场景
- NEVER 硬判定唯一源头：知识溯源只标注，不下结论
- 不给总分：Health Check 等所有评估类输出，给维度不给分数
- 冲突是高价值知识：标注冲突，不消解冲突
- 管线通用，知识领域特定（v5.2 新增）：Phase A-H 流程不随领域变化，变化的是搜索/提取/社区资源

B. 产品哲学术语规范

禁止用词	推荐用词
错误	观测到差异
漏掉	未覆盖
落后	非主流路径
错误共识	高风险共识
诊断	知识位置定位
健康/不健康	（不使用，给维度数据）
应该/必须（面向用户）	（不使用，给工具不给建议）

C. 关键数据结构速查（v5.2 更新）

结构	用途	首次出现
`project_fingerprint.json`	机器可比较的项目知识指纹	跨项目智能
`repo_facts.json`	确定性提取的项目事实	Soul Extractor v0.9
`hypothesis_list.json`	Agent 探索假说	Agentic 提取
`exploration_log.jsonl`	Agent 工具调用记录	Agentic 提取
`claim_ledger.jsonl`	已确认/已反驳知识声明	Agentic 提取
`health_check.json`	体检结果	知识健康检查
`deprecation_events.jsonl`	废弃事件数据库	STALE 检测
`domain_map.sqlite`	领域共识图谱	领域图谱
`DOMAIN_TRUTH.md`	领域真相报告	领域图谱
`HEALTH_CHECK.md`	项目体检报告	知识健康检查
`SOUL_DIFF.md`	项目对比报告	跨项目智能
`STITCH_MAP`	缝合可能性地图	StitchCraft
`need_profile.json`	用户需求结构化表示	多项目管线 Phase A（v5.2）

结构	用途	首次出现
`discovery_result.json`	作业发现结果（候选项目+搜索覆盖）	多项目管线 Phase B（v5.2）
`community_knowledge.json`	社区 skill/教程/用例的结构化知识	多项目管线 Phase D（v5.2）
`synthesis_report.md`	多源知识综合报告（共识+冲突+独创）	多项目管线 Phase E（v5.2）
`platform_rules.json`	OpenClaw 平台适配规则配置	平台适配 4.11（v5.2）
`validation_report.json`	多项目管线质量门控结果	多项目管线 Phase G（v5.2）